

LAPORAN TUGAS · BACKEND API

TokoJoko

Dokumentasi & Tutorial Backend REST API
E-Commerce (Toko Bunga) — Laravel 10 + Sanctum

Mata Kuliah	Pemrograman Perangkat Bergerak / Pemrograman Sisi Client
Nama Mahasiswa	_____
NIM	_____
Kelas	_____
Anggota Kelompok	_____
Repository	_____
Tanggal	12 Juli 2026

1 Ringkasan Proyek

TokoJoko adalah aplikasi e-commerce (toko bunga) berbasis Laravel 10. Aplikasi terdiri dari dua bagian:

- **Storefront & Admin Panel** — dibangun dengan Livewire 2 (halaman produk, keranjang, checkout) dan Filament 2 (panel admin), berjalan berbasis session/cookie.
- **REST API** (`/api/v1/*`) — layer backend stateless berbasis token yang didesain untuk dikonsumsi klien eksternal: aplikasi mobile (tugas PPB) atau aplikasi web/SPA sisi klien (tugas PSC).

Dokumen ini berfokus pada REST API tersebut dan menjelaskan bagaimana proyek memenuhi kedelapan ruang lingkup UAT backend yang ditentukan.

Komponen	Teknologi
Bahasa & Framework	PHP 8.1+ (diuji di PHP 8.4), Laravel 10.50
Autentikasi API	Laravel Sanctum (Personal Access Token / Bearer Token)
Database	MySQL 8 (aplikasi), SQLite in-memory (automated testing)
Testing	PHPUnit (bawaan Laravel) — 48 automated test, 24 di antaranya khusus API
Dokumentasi API	OpenAPI 3.0 + Swagger UI (<code>/docs</code>), Postman Collection, Markdown (<code>docs/API.md</code>)
Frontend admin	Filament 2 (resource: Product, Category, Order, User, BillingDetail)

2 Desain Database, Model & Schema

Skema database terdiri dari 6 tabel inti yang relevan dengan e-commerce, dikelola lewat Eloquent ORM dan migration Laravel.

Tabel	Kolom Utama	Relasi
<code>users</code>	name, email, password, is_admin	hasMany Order, hasOne BillingDetail
<code>categories</code>	name, slug	belongsToMany Product (pivot <code>category_product</code>)

<code>products</code>	name, description, price, old_price, SKU, stock_status, quantity, image, images	belongsToMany Category
<code>orders</code>	user_id, status, total, session_id	belongsTo User, hasMany OrderItem
<code>order_items</code>	order_id, product_id, quantity, price	belongsTo Order, belongsTo Product
<code>billing_details</code>	user_id, country, billing_address, city, state, zipcode, phone, order_notes	belongsTo User

Alur relasi singkat (ERD naratif):

```
User 1---1 BillingDetail
User 1---N Order
Order 1---N OrderItem N---1 Product
Product N---N Category (lewat pivot category_product)
```

Setiap model menggunakan `$fillable` untuk mass-assignment protection, dan foreign key di migration memakai `onDelete('cascade')` agar data anak (order_items, category_product) ikut terhapus saat parent-nya dihapus — menjaga integritas referensial.

3 API

Seluruh endpoint API berada di bawah prefix `/api/v1`, didaftarkan di `routes/api.php`, dan diproses oleh controller khusus di namespace `App\Http\Controllers\Api`.

Method	Endpoint	Auth	Keterangan
POST	<code>/api/v1/auth/register</code>	—	Registrasi user baru, langsung menerima token
POST	<code>/api/v1/auth/login</code>	—	Login, menerima bearer token
POST	<code>/api/v1/auth/logout</code>	✓	Mencabut token yang sedang dipakai
GET	<code>/api/v1/auth/me</code>	✓	Profil user yang sedang login
GET	<code>/api/v1/categories</code>	—	List kategori (pagination)
GET	<code>/api/v1/categories/{id}</code>	—	Detail kategori
GET	<code>/api/v1/products</code>	—	List produk (pagination, filter, search, sort)
GET	<code>/api/v1/products/{id}</code>	—	Detail produk
GET	<code>/api/v1/orders</code>	✓	List order milik user (admin melihat semua)

POST	/api/v1/orders	✓	Membuat order (checkout) + validasi stok
GET	/api/v1/orders/{id}	✓	Detail order (hanya pemilik/admin)

Contoh Request & Response

Login

```

POST /api/v1/auth/login
Content-Type: application/json

{
  "email": "jane@example.com",
  "password": "password123"
}

→ 200 OK
{
  "message": "Login successful.",
  "user": { "id": 3, "name": "Jane Doe", "email": "jane@example.com", "is_admin": false },
  "token": "1|gmJJpegoRFJFsv0q2s0psNp41svDc1vL3EKhgrBC9a78792b"
}

```

Membuat Order (checkout)

```

POST /api/v1/orders
Authorization: Bearer <token>
Content-Type: application/json

{
  "items": [ { "product_id": 1, "quantity": 2 } ],
  "billing": {
    "country": "Indonesia", "billing_address": "Jl. Mawar No. 1",
    "city": "Bandung", "state": "Jawa Barat", "zipcode": "40123", "phone": "081234567890"
  }
}

→ 201 Created
{
  "message": "Order created successfully.",
  "data": { "id": 4, "status": "pending", "total": 20000,
    "items": [ { "product_id": 1, "quantity": 2, "price": 10000, "subtotal": 20000 } ] }
}

```

4 Autentikasi (AUTH)

Autentikasi API menggunakan **Laravel Sanctum** dengan skema *Personal Access Token* (bukan cookie/session seperti storefront Livewire), sehingga API benar-benar stateless dan cocok dikonsumsi aplikasi mobile maupun SPA eksternal.

- `POST /auth/register` dan `POST /auth/login` mengembalikan token dalam bentuk `Bearer` string.
- Setiap endpoint privat (orders, logout, me) diproteksi middleware `auth:sanctum` dan mengharuskan header `Authorization: Bearer <token>`.
- Tanpa token → `401 Unauthenticated`. Token valid tapi mengakses resource orang lain → `403 Forbidden` (lihat `app/Policies/OrderPolicy.php`).
- `POST /auth/logout` mencabut token yang sedang dipakai (`currentAccessToken()->delete()`), sehingga token tidak bisa dipakai ulang.

5 Throttling, Pagination & Filtering

Throttling (Rate Limiting)

Cakupan	Batas	Response saat terlampaui
Seluruh <code>/api/*</code>	60 request/menit per user (atau per IP untuk tamu)	429 Too Many Requests
<code>/auth/register</code> , <code>/auth/login</code>	6 request/menit per IP (tambahan, lebih ketat)	429 Too Many Requests

Pagination

Endpoint list (`/products` , `/categories` , `/orders`) selalu mengembalikan struktur konsisten:

```
{
  "data": [ ... ],
  "meta": { "current_page": 1, "last_page": 3, "per_page": 12, "total": 30 }
}
```

Parameter `per_page` dibatasi maksimum 50 untuk mencegah query yang terlalu berat.

Filtering & Sorting (khusus `/products`)

Parameter	Contoh	Efek
category	<code>?category=mawar</code>	Filter berdasarkan slug kategori
search	<code>?search=rose</code>	Cari di kolom name & description
min_price / max_price	<code>?min_price=10&max_price=100</code>	Filter rentang harga
sort	<code>?sort=price_asc</code>	latest / price_asc / price_desc

6 Unit Testing

Automated testing memakai PHPUnit (bawaan Laravel), dijalankan di atas database **SQLite in-memory** (dikonfigurasi di `phpunit.xml`) sehingga tidak pernah menyentuh database development.

File Test	Cakupan
<code>tests/Feature/Api/AuthApiTest.php</code>	Register, login, validasi email duplikat, salah password, rate limit, me, logout (8 test)
<code>tests/Feature/Api/ProductApiTest.php</code>	Pagination, filter kategori, search, filter harga, sorting, detail, 404 (7 test)
<code>tests/Feature/Api/CategoryApiTest.php</code>	List, detail, 404 (3 test)
<code>tests/Feature/Api/OrderApiTest.php</code>	Guest ditolak, checkout sukses + stok berkurang, stok kurang → 422, isolasi order per-user, 403 order orang lain, admin bisa lihat semua (6 test)

Cara menjalankan:

```
php artisan test          # seluruh test suite (48 test)
php artisan test --filter=Api  # hanya 24 test API
```

Status hasil pengujian terakhir: **48/48 PASSED** (123 assertions, ~6-7 detik).

7 Dokumentasi

Tersedia tiga bentuk dokumentasi API, saling melengkapi:

1. **Markdown reference** — `docs/API.md` , deskripsi tiap endpoint beserta contoh request/response.

2. **Swagger UI interaktif** — buka `/docs` setelah server jalan (contoh: `http://localhost:8000/docs`). Menampilkan spesifikasi OpenAPI 3.0 dari `public/openapi.json`, lengkap dengan tombol **Authorize** untuk mencoba endpoint privat langsung dari browser.
3. **Postman Collection** — `docs/postman_collection.json`, siap diimpor, otomatis menyimpan token ke variable `{{token}}` setelah request Login/Register berhasil.

8 Tutorial Pengujian Manual (Postman & Swagger)

A. Menjalankan server backend

```
composer install
cp .env.example .env
php artisan key:generate
# set DB_DATABASE / DB_USERNAME / DB_PASSWORD di .env
php artisan migrate
php artisan serve
```

Server berjalan di `http://localhost:8000`.

B. Tutorial via Swagger UI (paling cepat, tanpa install apapun)

1. Buka browser, akses `http://localhost:8000/docs`.
2. Expand endpoint `POST /auth/register`, klik **Try it out**, isi body contoh, klik **Execute** → salin nilai `token` dari response.
3. Klik tombol **Authorize** di kanan atas, tempel token (tanpa perlu menulis ulang kata “Bearer”, Swagger menambahkannya otomatis), klik Authorize.
4. Sekarang endpoint privat (`/auth/me`, `/orders`) bisa langsung dicoba — klik Try it out → Execute pada masing-masing endpoint dan amati response.
5. Coba `POST /orders` dengan body items + billing sesuai contoh di spesifikasi, lalu verifikasi dengan `GET /orders` bahwa order baru muncul.

C. Tutorial via Postman

1. Buka Postman → **Import** → pilih file `docs/postman_collection.json`.
2. Set collection variable `base_url` ke `http://localhost:8000/api/v1` (tab *Variables* pada collection).
3. Jalankan request **Auth** → **Register** (atau Login jika user sudah ada) — token otomatis tersimpan ke variable `{{token}}` lewat test script bawaan collection.

4. Jalankan **Products** → **List products**, ubah query string (`category` , `search` , `sort`) untuk menguji filtering.
5. Jalankan **Orders** → **Create order** (memakai `{{product_id}}`), lalu **Orders** → **Show order** untuk memverifikasi.
6. Uji skenario negatif: hapus header Authorization pada request Orders → harus dapat `401` ; kirim `quantity` melebihi stok → harus dapat `422` dengan pesan stok tidak cukup.

Seluruh skenario di atas (sukses maupun gagal/edge case) sudah diverifikasi berjalan benar baik lewat automated test (PHPUnit) maupun pengujian manual langsung ke server yang sedang berjalan (curl end-to-end: register → login → checkout → validasi stok → proteksi kepemilikan).

9 Hosting

- Sebelum deploy: set `APP_ENV=production` , `APP_DEBUG=false` , dan `APP_URL` sesuai domain final.
- `config/cors.php` saat ini mengizinkan semua origin (`allowed_origins => ['*']`) agar mudah diuji dari Postman/aplikasi mobile selama pengembangan & penilaian — persempit ke origin klien asli untuk produksi.
- API bersifat stateless (token, bukan session), sehingga dapat di-deploy terpisah dari storefront Livewire jika diperlukan (misalnya di layanan seperti Railway, Render, atau shared hosting yang mendukung PHP 8.1+ & MySQL).

10 Kesimpulan — Pemenuhan Ruang Lingkup

✓	Desain database, model & schema — migration, model, relasi Eloquent lengkap
✓	API — REST API stateless <code>/api/v1/*</code> untuk auth, produk, kategori, order
✓	Auth — Sanctum bearer token, register/login/logout/me
✓	Throttling, pagination, filtering — rate limit 60/menit (+6/menit di auth), pagination konsisten, filter & sort produk
✓	Unit testing — 48 automated test (24 khusus API), semua PASSED
✓	Docs — Markdown, Swagger UI (<code>/docs</code>), Postman Collection
✓	Uji minimal Postman/aplikasi kecil — tutorial & collection tersedia, sudah diverifikasi end-to-end



Hosting — konfigurasi produksi (env, CORS) terdokumentasi, API stateless siap deploy

Dokumen ini dihasilkan sebagai bagian dari laporan tugas backend TokoJoko. Kode sumber lengkap tersedia di repository proyek; lihat [README.md](#) untuk instruksi setup lengkap.